

Formatos

❖ BLOQUE :.

☒ :

- :

Portada y Información adicionales

PARADIGMAS Y LENGUAJES

TRABAJO INTEGRADOR

Encabezado 1----- 2

Encabezado 1

INFORMACIÓN		
PRÁCTICO:	Integrador	
TÍTULO:	Semaforo inteligente.	
Realizado desde:	2026	Integrantes:
Docente:		• Zarate Lucas Martin • - Portillo Joel • - Quinteros Agustín Emiliano • - Meza Lautaro Gabriel

Bloque 1: Analisis del Proyecto

❖ BLOQUE 1: Análisis del proyecto.

☒ Objetivo del Bloque:

Analizar los documentos fuente de la cátedra, establecer los principios fundamentales del paradigma funcional y justificar técnicamente el uso de common lisp bajo restricciones de inmutabilidad absoluta y programación declarativa.

☒ OBJETIVOS TÉCNICOS Y FUNCIONALES:

- Modelado de la lógica urbana:

Diseñar un sistema capaz de controlar el flujo de semáforo en intersecciones críticas utilizando:

- Estados lógicos
- Temporizador basado en tiempo Unix/EPoch.
- Transiciones controladas entre colores y etapas del tráfico.

- Optimización del tráfico:

Analizar métricas del funcionamiento de los semáforos, como:

- Tiempo de asignación a cada luz.
- Porcentaje de pertenencia en cada estado.
- Transiciones controladas entre colores y etapas del tráfico.

El objetivo es manejar la circulación y facilitar auditorías del tránsito.

- Robustez y Persistencia:

Implementar mecanismo que permitan:

- Mantener el sistema operativo incluso ante errores o fallas.
- Registrar de forma permanente los cambios de estado y transiciones.
- Guardar información en archivos físicos mediante operaciones de entrada/salida en common lisp.

☒ Introducción al Paradigma Funcional:

- Paradigma funcional y programación declarativa:

Es un modelo de programación declarativa basado en la composición matemática de funciones. A diferencia del paradigma imperativo, donde el programa modifica estados internos mediante secuencias de instrucciones, el enfoque funcional prioriza la transformación de datos inmutables mediante funciones puras.

O sea, cada función recibe datos de entradas y devuelve nuevos resultados sin alterar estructuras externas ni producir efectos secundarios ocultos, permitiendo desarrollar sistemas más predecibles, testeables y seguros para escenarios concurrentes.

- Filosofía minimalista Funcional en Scheme:

Scheme propone construir sistemas mediante herramientas simples y componibles (significa que pequeñas partes simples del programa pueden combinarse entre sí para construir comportamientos más grandes y complejos, o sea, definir varias funciones que hagan una acción y luego convocarlas. Esto facilita probarlas y utilizarlas.), de esta forma se evita estructuras innecesariamente complejas.

En lugar de depender de:

- Variables locales y mutables.
- Bucles imperativos clásicos.
- Objetos con estado interno modificable.
- Alteraciones destructivas de la memoria.

En vez de eso se construye mediante:

- Paso de argumentos.
- Retorno de nuevos estados.
- Transformaciones funcionales puras.
- Composición de funciones.

☒ Restricciones Funcionales del proyecto:

El proyecto prohíbe el uso de ciertas características de Common Lisp para preservar transparencia y la inmutabilidad del sistema.:

- `setq`,
- `setf`
- Variables globales mutables.
- Operaciones destructivas.
- Bucles imperativos tradicionales.

Como consecuencia, toda transición de estado debe implementarse mediante funciones puras capaces de recibir un estado y devolver uno nuevo sin modificar el original.

☒ Modelo de Estado Inmutable

- Diferencia entre mutabilidad y inmutabilidad:

Al programar en lenguajes orientados a objetos (**como Python, Java o C++**), se trabaja con objetos mutables. Esto significa que el mismo objeto puede cambiar internamente durante la ejecución del programa. Por ejemplo, un semáforo puede representarse mediante una clase con un atributo llamado color:

Semaforo.color = "ROJO"

Más adelante el programa modifica ese mismo objeto:

Semaforo.color = "VERDE"

En este caso, el estado original "ROJO" desaparece y es reemplazado por "VERDE" dentro de la misma estructura de memoria. Es decir el objeto fue alterado.

Si un semáforo está en 'rojo, ese valor permanece inactivo. Para cambiarlo se genera un nuevo estado independiente:

(transicion 'rojo) => 'verde

De esta manera:

- El estado anterior permanece intacto,
- No existen cambios ocultos en la memoria,
- Y el sistema se vuelve más predecible y seguro.

- Importante de la inmutabilidad en sistema concurrentes:

La importancia de este diseño aparece especialmente en sistemas concurrentes. En un entorno urbano real, múltiples semáforos pueden ejecutarse simultáneamente. Si varios procesos modifican la misma estructura mutable al mismo tiempo, pueden producirse:

- Errores de sincronización
- Corrupción de estados compartidos.
- Inconsistencia lógica.

Al trabajar con estados inmutables, cada estado genera el siguiente mediante transformaciones puramente funcionales. Esto garantiza un comportamiento donde la misma entrada siempre produce la misma salida.

- Ventajas del modelo inmutable:

- El comportamiento del sistema es totalmente predecible.
- Se reducen los errores de sincronización.
- Se evita la corrupción de estados compartidos.
- El sistema es más seguro para entornos concurrentes.

- Comparación: Objetos mutables vs datos inmutables.

Característica	Orientado a Objetos	Funcional Puro
Representación	Objetos mutables con atributos internos	Datos simples e inmutables
Modificación	Mutación directa de memoria. Ejemplo: <code>semaforo.cambiarColor('verde')</code>	Generación de nuevos estados. Ejemplo: <code>(transicion 'en-rojo' 'verde') => 'en-verde'</code> .
Estados ocultos	Los atributos internos pueden modificarse y que otras partes del sistema no lo detecte inmediatamente	El estado se pasa explícitamente como argumentos entre funciones puras
Estado secundarios	Una modificación puede afectar otras partes del sistema que compartan la misma referencia del objeto.	Las funciones no alterarán los datos de entrada evitando cambios inesperados en el sistema.
Concurrencia	El acceso simultáneo a objetos mutables puede producir errores de sincronización.	Los datos inmutables reducen riesgos de sincronización y facilitan la ejecución concurrente
Predictibilidad	El comportamiento puede variar dependiendo del orden y momento en que ocurran las mutaciones	La misma entrada siempre genera la misma salida, garantizando un comportamiento determinista.

- Explicaciones técnicas:

El sistema representa los estados de tráfico usando datos simples e inmutables. Cada estado funciona como un momento específico del semáforo. La idea principal es tratar los estados como valores matemáticos. Por ejemplo, si un semáforo está **'en-rojo'**, ese valor no cambia internamente. Cuando ocurre una transición, se genera un nuevo estado en lugar de modificar el anterior.

Este enfoque evita errores comunes en sistemas concurrentes, especialmente problemas de sincronización o estados corruptos causados por múltiples procesos trabajados al mismo tiempo sobre la misma estructura de datos.

Por esta razón, se descartó el uso de CLOS (Common Lisp Object System). Aunque permite crear slots mutables mediante operaciones como `setf`, lo cual rompe con las restricciones de la cátedra.

Como ventaja, el sistema obtiene un comportamiento mucho más predecible y fácil de depurar. Sin embargo, este modelo también obliga a cambiar la forma tradicional de programar, ya que el paso del tiempo debe representarse mediante recursión y transformación de estados en lugar de bucles imperativos clásicos.

```

TP_Integrador_Gru...
25 ;;;; *****
24 ;;;; Archivo: modelado_arquitectura.lisp
23 ;;;; Modulo: Clarificación de Arquitectura - Grupo 29
22 ;;;; *****
21
20 ;;; =====
19 ;;; ENFOQUE NO RECOMENDADO (Orientado a Objetos / Mutable)
18 ;;; =====
17 ;;; Modifica el estado interno oculto de una variable global.
16 ;;; Esto viola la inmutabilidad absoluta del TPI.
15
14 (defvar *semaforo-esquina-1* 'en-rojo)
13
12 (defun cambiar-luz-imperativo (nuevo-color)
11 |   ;; MAL: Uso de setq destructivo que altera el estado
10 |   ;;     oculto global
9 |
8 |   (setq *semaforo-esquina-1* nuevo-color))
7
6 ;;; =====
5 ;;; ENFOQUE CORRECTO (Funcional Puro / Datos Inmutables)
4 ;;; =====
3 ;;; El semáforo es solo un dato. La función no altera
2 ;;; nada externo;
1 ;;; recibe el estado anterior y proyecta el nuevo estado
26 ;;; en una nueva lista.
1 (defun transicion (color-actual cambiar-a)
2 |   "Recibe simbolos puros y retorna una nueva"
3 |   " estructura de lista con la accion."
4 |   (cond
5 |     ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde))
6 |      (list 'en-verde "cambiar-a-verde"))
7 |     ((and (eq color-actual 'en-verde) (eq cambiar-a 'amarillo))
8 |      (list 'en-amarillo "cambiar-a-amarillo"))
9 |     ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'rojo))
10 |      (list 'en-rojo "cambiar-a-rojo"))
11 |     ;; Comportamiento por defecto: transición inválida
12 |     (t (list color-actual "accion-por-defecto"))))

```

- Elección Tecnológica del Proyecto:

- **Common lisp como nucleo logico:**

Lenguaje principal durante las fases 1 y 2 del proyecto. Se utiliza debido a:

- Su capacidad para manipular símbolos y listas de forma eficiente..
- Su compatibilidad con programación funcional.
- El uso del entorno RELP para pruebas rápidas y simulación del sistema.
- Facilidad para implementar recursos y composición funcional.

Aunque common lisp permite programación imperativa y orientada a objetos mediante CLOS, dicha capacidad será restringida para respetar las reglas funcionales establecidas por la cátedra.

- **Quicklisp:**

Se utilizará únicamente si es necesario incorporar:

- Librerías externas.
- Herramientas avanzadas de testing.
- Funciones de formateo adicionales.

Aun así, se prioriza el uso de funciones nativas para mantener el enfoque minimalista requerido.

☒ **Contraste Técnico entre Scheme y Common Lisp (Lisp-1 vs Lisp-2):**

Una de las diferencias más notables entre ambos lenguajes es la forma en que administran funciones y variables:

- **Scheme:** Es un lisp 1 (un solo espacio de nombre). En otras palabras tiene un cajón y este es grande, al guardar una carpeta llamada transición este puede contener un número, un texto o código de una función. Pero ese nombre de transición está ocupado en ese único cajón. No se puede tener una variable y una función con el mismo nombre al mismo tiempo, porque se mezclarán.
- **Common lisp:** Es un lisp 2 (tienen espacios de nombre separados por funciones y variables). En esta situación es diferente el mueble en vez de tener un cajón se tiene dos cajones para cada nombre. Un cajón es para la variable y el otro para la función por lo que llamarlas de la misma forma el lenguaje no los confunde.

Como **common lisp** tiene dos cajones cuando se quiera pasar una función como argumento a otra función (como cuando se usa mapcar para actualizar una lista de semáforos – – por ejemplo), si se escribe transición solamente, lisp va a ir a buscar por defecto al cajón de la variable. Para obligarlo a abrir el cajón de las funciones y sacar el código, se tiene que poner el prefijo #' que es una etiqueta que le dice al lenguaje “busca el cajón de funciones, no en el cajón de variables”, en

otras palabras es un atajo sintáctico, escribir `#'transición` es como escribir `function transición`. O sea podemos hacer lo siguiente:

```
(let ((transicion '(rojo verde))) (ejecutar #'transicion transicion))
```

En **Scheme**, como solo hay un cajón, las funciones se tratan exactamente igual que cualquier variable. No hace ningun simbolo, por eso que en scheme las estrategias de orden superior quedan visualmente más limpias (las funciones se pasan como parámetros)

☒ Evaluación Funcional y Funciones de Orden Superior:

- Explicación técnica (El Operador Sharp-Quote (`#'`)):

Debido a la arquitectura Lisp-2, requiere mecanismos explícitos para recuperar funciones como datos.

El operador `#'` indica al evaluador que debe acceder al espacio funcional asociado a un símbolo.

Ejemplo:

```
(mapcar #'transicion lista-semáforos)
```

Sin el uso del operador el sistema buscará el símbolo dentro del espacio de variables, produciendo un error que dará `relp, execution of uncompiled code: TRANSICION is unbound as a variable`.

Código: Diferencia Práctica de Sintaxis

A continuación se muestra cómo se codifica el mismo comportamiento de orden superior (aplicar una función de transición a una lista de estados) en ambos mundos:

1. Enfoque Common Lisp

```

TP_Integrador_Gru... ●
12 ;;;; *****
11 ;;;; Fichero: espacios_nombres.lisp
10 ;;;; Modulo: Demostración Lisp-2
9 ;;;; *****
8
7 (defun simular-cambio (funcion-controlador estado)
6   "Recibe una función en el primer parámetro y la"
5   "ejecuta usando funcall"
4   (funcall funcion-controlador estado 'verde))
3
2 ;; Invocación Correcta:
1 ;; Usamos #' porque queremos pasar la FUNCIÓN
13 ;; 'transicion-segura', no una variable.
1 (simular-cambio #'transicion-segura 'en-rojo)
2

```

2. Enfoque Scheme (Lisp-1)

```

TP_Integrador_Gru... ●
8 ;;;; Código equivalente en Scheme (Para comparar)
7 (define (simular-cambio funcion-controlador estado)
6   ;; No requiere funcall; la función se invoca
5   ;; directamente poniendo el parámetro al inicio
4   (funcion-controlador estado 'verde))
3
2 ;; Invocación en Scheme:
1 ;; Se pasa directamente el nombre de la función,
1 ;; sin #' ni caracteres especiales
1 (simular-cambio transicion-segura 'en-rojo)
2

```


- Evaluación dinámica mediante FUNCALL :

Cuando una función es recibida como argumento por otra función, common lisp requiere utilizar funcall para ejecutar dinámicamente el objeto funcional almacenado en una variable local.

Su sintaxis es estrictamente posicional y se ubica siempre en el cuerpo interno de nuestras funciones de orden superior:

```
(funcall regla estado)
```

En este caso, regla contiene una función y el estado es el argumento que recibe.

Sin funcall, el intérprete intentará resolver el símbolo como una función global, arrojando un error como Undefined function: REGLA.

Porque se puede llegar a usar? Common Lisp separa las variables y las funciones en espacios distintos. Por eso, cuando una función está guardada dentro de una variable, no puede ejecutarse directamente; primero debe invocarse con funcall.

☒ Optimización Funcional y Recursión de cola:

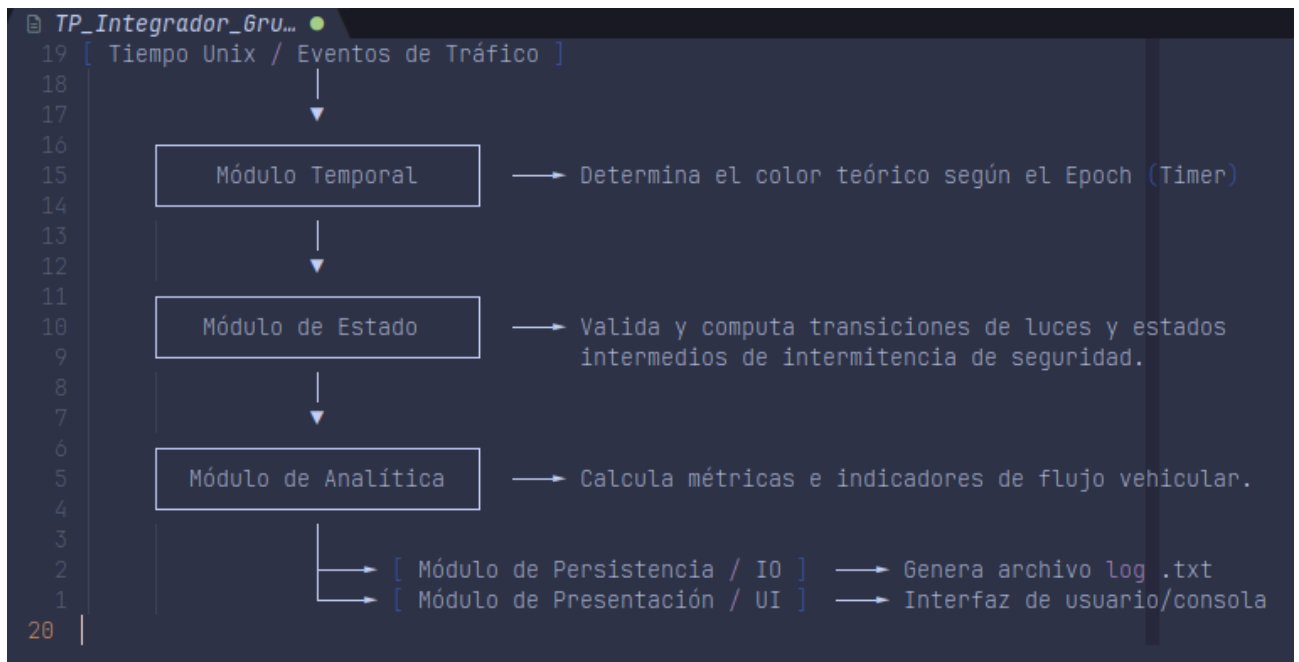
- Arquitectura funcional preliminar:

DEbido a las restricciones de inmutabilidad, el simulador deberá estructurarse mediante tuberías de transformación funcional donde cada módulo reciba un estado y devuelva uno nuevo sin alterar estructuras externas.

De manera conceptual, el módulo del sistema puede dividirse en:

- Módulo temporal
- Módulo de estados
- Modulo analítico
- persistencia e I/O controlado.

Cada uno trabaja mediante composición funcional y paso explícito de datos.



- Separación entre Lógica pura y efectos secundarios:

La arquitectura propuesta distingue dos capas principales:

Capa funcional pura:

Compuesta por funciones encargadas de:

- Transiciones.
- Cálculo temporal.
- Reglas del tráfico.
- Transformación del estado.

Capa de efectos secundarios (I/O):

Compuesto por funciones encargadas de:

- Persistencia en archivos
- Impresión por consola
- Formateo visual
- Registro de eventos

De esta forma los efectos secundarios quedan aislados de la lógica funcional principal.

- Recursión de cola:

Debido a las restricciones funcionales del proyecto, el flujo principal del simulador deberá constituirse mediante recursión de cola en lugar de bucles imperativos.

Una función posee recursión de cola cuando la llamada recursiva constituye la última operación ejecutada.

Recursión no optimizada:

```
(defun contar-malo (n)
  (if (= n 0)
      0
      (+1(contar-malo(- n 1)))))
```

En este caso, la llamada recursiva `(+1(contar-malo(- n 1)))` no es la última operación, porque después todavía debe ejecutarse `(+1 ...)`, esto obliga al intérprete a mantener información pendiente en memoria.

Recursión de cola:

```
(defun contar-bueno (n acumulador)
  (if (= n 0)
      acumulador
      (contar-bueno(- n 1) (+ acumulador 1)))))
```

el llamado inicial (contar-bueno 5 0), aquí la llamada recursiva (contar-bueno(- n 1) (+ acumulador 1)) es la última operación de la función en ejecutarse y no queda cálculos operativos colgados o pendientes. Porque la recursión de cola permite contruir procesos repetitivos de manera más eficiente y compatible con el paradigma funcional que utiliza common lisp.

- Tail Call Optimization (TCO):

Scheme garantiza por estándar la optimización de llamadas de cola.

Common Lisp depende de las optimizaciones implementadas por el compilador utilizado (SBCL o CLISP).

La optimización permite reutilizar el mismo espacio de memoria, evitando crecimiento innecesario de pila y reduciendo el riesgo de stack overflow.

Bloque 2: Requisitos Funcionales y Backend

❖ BLOQUE 2 Requisitos Funcionales y Backend.

☒ Objetivo del bloque:

Desarrollar el núcleo lógico del sistema de semáforo utilizando estructuras funcionales puras, símbolos atómicos, temporización basada en tiempo Unix Epoch, aritmética modular, estado de intermitencia de seguridad, cálculo de métrica de flujo vehicular y persistencia física no destructiva..

☒ Modelo Simbólico funcional de estados:

- Representación funcional de estados:

Para representar las luces de semáforo sin recurrir variables mutables (**es una entidad o espacio de memoria cuyo valor o estado asociado puede ser alterado o modificado a lo largo del tiempo de ejecución - que desde el punto de vista de paradigma no es deseable ya que rompe la transparencia referencial, o sea que la función devuelve siempre el mismo resultado para los mismos argumentos. Esto tiene una excepción en el uso de let para E/S**) u objetos complejos (**es una estructura como en c**).

Este enfoque permite modelar cada estado como un dato inmutable e independiente, preservando la transparencia referencial del sistema:

- **'en-rojo:** Foco rojo encendido. Detención absoluta del tráfico.
- **'en-verde:** Foco verde encendido. Permiso de circulación activa.
- **'en-amarillo:** Foco amarillo encendido. Fase de desaceleración y advertencia.
- **'amarillo-intermitente: (Extencion 1 de la interacción 2)** Estado transicional de seguridad activa antes del cambio total de fase vial.

Ventajas de los símbolos dentro del paradigma funcional:

- Son livianos y poseen identidad única.
- Facilita comparaciones mediante eq.
- Respetan estados semánticos de manera clara y evita estructuras mutables complejas.

☒ Diseño de la Funcion de Transicion:

- Objetivo funcional:

La función transición constituye el núcleo lógico principal del simulador urbano. Su responsabilidad consiste en validar cambios de estado, determinar acciones operativas y mitigar caminos inválidos.

- Estructura de entrada:

Las funciones reciben dos parámetros simbólicos:

Parámetro	Tipo	Descripción
color-actual	símbolo	Estado actual del semáforo
cambiar-a	símbolo	Estado destino solicitado

Ejemplo de invocación: (transicion 'en-rojo 'verde)

- Estructura de salida:

La función devuelve una lista evaluada compuesta por:

(estado-actual "acción-literal")

Ejemplo de salida: ('en-rojo "cambiar-a-verde")

La salida no devuelve directamente el nuevo estado, sino el estado actual y la acción operativa que deberá ejecutarse posteriormente.

☒ Reglas de transiciones:

- Transiciones Válidas:

Estado actual	Destino	Resultado
'en-rojo	'verde	("cambiar-a-verde")
'en-verde	'amarillo	("cambiar-a-amarillo")
'en-amarillo	'rojo	("cambiar-a-rojo")
'en-verde	'amarillo-intermitente	("cambiar-a-amarillo-intermitente")
'amarillo-intermitente	'Rojo	("cambiar-a-rojo")
Cualquier otro	Cualquier otro	(list color-actual "accion-por-defecto")

- Transiciones inválidas:

Cualquier combinación no contemplada explícitamente será tratada como un camino alternativo de seguridad. Resultado: (list color-actual "acción-por-defecto")

☒ Implementación Funcional:

- Implementación de transición:

```
(defun transicion (color-actual cambiar-a)

  (cond

    ((and (eq color-actual 'en-rojo)
          (eq cambiar-a 'verde))
      (list color-actual "cambiar-a-verde"))

    ((and (eq color-actual 'en-verde)
          (eq cambiar-a 'amarillo))
      (list color-actual "cambiar-a-amarillo"))

    ((and (eq color-actual 'en-amarillo)
          (eq cambiar-a 'rojo))
      (list color-actual "cambiar-a-rojo"))

    ((and (eq color-actual 'en-verde)
          (eq cambiar-a 'amarillo-intermitente))
      (list color-actual "cambiar-a-amarillo-intermitente"))

    ((and (eq color-actual 'amarillo-intermitente)
          (eq cambiar-a 'rojo))
      (list color-actual "cambiar-a-rojo"))

    (t
      (list color-actual "acción-por-defecto"))))
```


☒ Evaluación Dinámica y Construcción de listas:

- Problema de acotación estática:

Durante las pruebas realizadas en el entorno RELP se observó un problema asociado al uso incorrecto del apóstrofe (') sobre listas completas.

Ejemplo incorrecto: '(color-actual "cambiar-a-verde")

Resultado: (COLOR-ACTUAL "cambiar-a-verde")

En este caso, el símbolo color-actual no era evaluado dinámicamente.

- Explicación Técnica:

En common lisp, el apóstrofo indica que toda la estructura debe tratarse como datos literales sin evaluación. Por esta razón, las variables internas quedan congeladas simbólicamente.

- Solución mediante list:

La solución consiste en utilizar la función constructora list:

(list color-actual "cambiar-a-verde")

Esto permite:

- Evaluar variables dinámicas.
- Combinar datos fijo y calculados
- Preservar comportamiento funcional correcto.

☒ Errores frecuentes en el RELP:

- Variables no declaradas:

Uno de los problemas más comunes es pasarle a la función variables que no están declaradas como por ejemplo (transicion en-rojo verde) esto produce un error que da el siguiente mensaje Unbound variable, porque el common lisp intenta evaluar los símbolos como variables activas. De esta forma para poder corregirla usamos el apostrofe transformando el simbolo en un dato literal, osea, (transicion 'en-rojo 'verde)

- Confusión entre símbolos y cadenas:

Otros de los problemas al querer probar una función es pasar una cadena en vez de un dato, la causa de este conflicto es la forma que se toma como comparación de identidad simbólica y no como contenido textual, o sea, se verá que en el código se usa un eq y no un equal para la comparación. La corrección para este problema es justamente el uso del apostrofe (transicion 'en-rojo 'verde)

- Confusión de espacios de nombres (Lisp-2):

Cuando se quiere consultar una función en RELP el error que puede generar al simplemente poner transición es que arroja un error de variable no ligada, ya que no accedemos a la función que fue declarada sino una variable que no fue definida, esto es así porque a diferencia de scheme que no tienen este problema es que common lisp tiene dos celdas de memoria para la función y la variable, dando la ventana de llamada a la función y a la variable de la misma forma, pero el llamado debe hacerse haciendo uso de '#' para decirle a common lisp que lo que se está llamando es una función.

☒ Extensión de intermitencia de Seguridad:

- Estado de intermitencia:

La segunda iteración del proyecto introduce estados de seguridad intermedios mediante intermitencia temporal.

Nuevo estado: 'amarillo-intermitente

Este estado representa una fase preventiva previa al retorno hacia rojo, permitiendo una desaceleración gradual y mayor seguridad vial antes de la detención completa del tráfico.

El modelo implementado incorpora una única fase intermitente simplificada con el objetivo de mantener compatibilidad con la arquitectura funcional y evitar complejidad excesiva en las reglas de transición.

De esta manera, el flujo principal del sistema queda definido conceptualmente como:

```
'en-rojo
  ↓
'en-verde
  ↓
'en-amarillo
  ↓
'amarillo-intermitente
  ↓
'en-rojo
```

Esto obliga a:

- actualizar reglas de transición
- extender validaciones
- modificar temporizadores
- adaptar métricas temporales
- mantener compatibilidad funcional

Bloque 3: Temporalizacion y Intermitencia

❖ BLOQUE 3: Lógica temporal, métricas de eficiencia e intermitencias de seguridad.

☒ Objetivo:

Consiste en diseñar e implementar, bajo el paradigma funcional puro (inmutabilidad absoluta y composición de funciones), la lógica matemática para el cálculo y validación de los ciclos viales. Se busca evaluar la eficiencia de la distribución de tiempos de las luces (focos) mediante la proyección de su comportamiento en una hora continua de análisis (3600 segundos) y estructura el sistema para asimilar la simulación basada en marcas de tiempo (Unix Epoch) junto con los estados intermedios obligatorios de seguridad viales (amarillo-intermitente).

☒ Desarrollo:

- Especificación de Requisitos y contratos de datos :

Para cumplir estrictamente a las especificaciones técnicas del proyecto, se adoptan y validan las constantes operativas inmutables:

Rango de Ciclo Vial Ideal: 35 segundos $\leq$ Duración Total $\leq$ 150 segundos

Período de Proyección Logística: 1 hora de análisis continuo (3600 segundos)

El estado de configuración de los tiempos de un semáforo se representa mediante una lista posicional simple indexada por orden fijo. La estructura formal elegida para la configuración de tiempos es una lista de tres elementos numéricos:

'(tiempo-rojo tiempo-amarillo tiempo-verde) -> '(45 5 40)

Donde:

- El primer elemento representa el tiempo en ROJO
- El segundo elemento representa el tiempo en AMARILLO.
- El tercer elemento representa el tiempo en VERDE.

- Diseño de Selectores Funcionales Puros:

Estas funciones tienen la tarea única de extraer el tiempo de cada luz a partir de la lista de configuración posicional.

```
(defun obtener-tiempo-rojo (configuracion)
```

```
;;; Naturaleza: Pura
```

```
;;; Entrada: Lista posicional '(rojo amarillo verde)
```

```
;;; Salida: Entero que representa los segundos del foco rojo  
(first configuracion))
```

```
(defun obtener-tiempo-amarillo (configuracion)
```

```
;;; Naturaleza: Pura
```

```
;;; Entrada: Lista posicional '(rojo amarillo verde)
```

```
;;; Salida: Entero que representa los segundos del foco amarillo  
(second configuracion))
```

```
(defun obtener-tiempo-verde (configuracion)
```

```
;;; Naturaleza: Pura
```

```
;;; Entrada: Lista posicional '(rojo amarillo verde)
```

```
;;; Salida: Entero que representa los segundos del foco verde  
(third configuracion))
```

Explicaciones Técnica:

Evitamos el uso de manejo de celdas cons del tipo car, cdr o caddr, ya que el uso de first, second y third actúa como una capa de abstracción auto documentada que aísla al resto del programa de la estructura interna de las listas.

- Diseño de la función duración y recomendación de ciclo :

Calcula la duración total de la suma de los tres focos básicos y determina si el ciclo se encuentra en el rango óptimo para la ingeniería de tráfico urbano, devolviendo un diagnóstico:

```
(defun duracion-ciclo (configuracion)
```

```
;;; Naturaleza: Pura
```

```
;;; Suma las componentes temporales de la lista posicional
```

```
(+ (obtener-tiempo-rojo configuracion)  
   (obtener-tiempo-amarillo configuracion)  
   (obtener-tiempo-verde configuracion)))
```

```
(defun recomendacion-ciclo (configuracion)
```

```
;;; Naturaleza: Pura. Evalúa los rangos ideales (35 a 150 segundos)  
total (let (((duracion-ciclo configuracion)))
```

```
(cond
  ((< total 35) 'ciclo-muy-corto-ajustar-a-alta-densidad)
  ((> total 150) 'ciclo-muy-largo-riesgo-de-congestion)
  (t 'ciclo-optimo-flujo-vehicular-eficiente))))
```

- Auditoría y Optimización de métricas :

Inicialmente se propuso calcular la proyección horaria mediante restas de contadores recursivos:

```
(defun contador-rojo (hora)
  (cond
    ((<= hora 0) 0)
    ((< hora 93) hora)
    ((> hora 0) (+ 93 (contador-rojo (- hora 225))))))
```

Puntos críticos:

- **Riesgos críticos de stack Overflow:** Para proyectar una hora completa (3600 segundos) o ciclos de simulación extensos, restar linealmente el tamaño del ciclo (225) obliga al intérprete a acumular múltiples llamadas recursivas en la pila de memoria. Al no ser una recursión de cola (Tail Recursion), satura la pila rápidamente.
- **Complejidad Algorítmica Ineficiente:** El costo de ejecución temporal es lineal con respecto al tiempo simulado, lo cual no puede ser aplicado en un sistema en tiempo real.
- **Valores quemados:** Los números como 93 y 225 están incrustados directamente dentro del código, violando la modularidad y el contrato de las listas posicional.

Optimización Funcional y Solución Analítica:

Se reescribe la métrica utilizando álgebra directa en tiempo constante $O(1)$. Dado que el semáforo es un sistema cíclico, la proporción ocupada por cada foco en una hora es idéntica a su porcentaje de ocupación dentro de un solo ciclo aislado. Esto suprime la recursión lineal y elimina el riesgo de desbordamiento de memoria. (40 5 45)

```
(defun calcular-porcentajes-eficiencia (configuracion)
  ;; Naturaleza: Pura
```

```

;;; Complejidad Algorítmica:  $O(1)$  - Tiempo Constante.
;;; Evita el Stack Overflow al remover por completo la recursión lineal.
(let* ((r (obtener-tiempo-rojo configuracion))
      (a (obtener-tiempo-amarillo configuracion))
      (v (obtener-tiempo-verde configuracion))
      (total (duracion-ciclo configuracion)))
  (list 'rojo (float (* (/ r total) 100))
        'amarillo (float (* (/ a total) 100))
        'verde (float (* (/ v total) 100)))))

```

Análisis del entorno léxico local:

- **Naturaleza de r, a, v y total:** Constituyen identificadores léxicos temporales definidos en el bloque `let*`. Representan los valores inmutables en segundos extraídos de la configuración (r para rojo, a para amarillo, v para verde) y lo suma acumulada de los mismos (total).
- **Preservación de la pureza funcional:** Su uso no viola las restricciones de inmutabilidad del paradigma declarativo, ya que actúa exclusivamente como un “ligado léxico”. A diferencia del paradigma imperativo, no modifica la celdas ni mutan posiciones de memoria a lo largo del tiempo de ejecución, garantizando la permanencia de la transparencia referencial en el cálculo analítico.

- Diseño del temporizador Adaptativo por Timestamp:

Determinar el color activo del semáforo en cualquier segundo absoluto del tiempo global (marca Unix Epoch) usando aritmética modular.

```

(defun timer (time-exac configuracion)
  ;;; Naturaleza: Pura
  ;;; Entrada: Entero largo (Timestamp), Lista de configuración posicional
  ;;; Salida: Símbolo del color activo ('rojo, 'amarillo, 'verde)
  (let* ((t-rojo (obtener-tiempo-rojo configuracion))
        (t-amarillo (obtener-tiempo-amarillo configuracion))
        (ciclo-total (duracion-ciclo configuracion))
        ;; Se obtiene la posición exacta dentro del ciclo actual usando
        módulo
        (segundo-actual (mod time-exac ciclo-total)))
    (cond
      ((< segundo-actual t-rojo) 'rojo)
      ((< segundo-actual (+ t-rojo t-amarillo)) 'amarillo)

```



```
(t 'verde))))
```

- Diseño del Temporizador de Seguridad (Intermitencia de 3 segundos):

Se extiende del estado operativo agregando una ventana fija de contingencia preventiva de 3 segundo (amarillo-intermitente) antes de cada transición entre luces principales:

```
(defun timer-seguro (time-exac configuracion)
  ;; Naturaleza: Pura
  ;; Entrada: Entero largo (Timestamp), Lista de configuración posicional
  ;; Salida: Símbolo del color activo ('rojo, 'amarillo-intermitente, 'amarillo,
  'verde)
  (let* ((t-rojo (obtener-tiempo-rojo configuracion))
        (t-amarillo (obtener-tiempo-amarillo configuracion))
        (t-verde (obtener-tiempo-verde configuracion))
        ;; Se añaden 3 segundos de intermitencia de seguridad en cada una
        de las 3 transiciones del ciclo
        (ciclo-total (+ t-rojo t-amarillo t-verde 9))
        (segundo-actual (mod time-exac ciclo-total)))
    (cond
      ((< segundo-actual t-rojo) 'rojo)
      ((< segundo-actual (+ t-rojo 3)) 'amarillo-intermitente)
      ((< segundo-actual (+ t-rojo 3 t-amarillo)) 'amarillo)
      ((< segundo-actual (+ t-rojo 6 t-amarillo)) 'amarillo-intermitente)
      ((< segundo-actual (+ t-rojo 6 t-amarillo t-verde)) 'verde)
      (t 'amarillo-intermitente))))
```

☒ Errores Frecuentes en el RELP durante este bloque:

Ejecución limpia que demuestra el funcionamiento correcto:

```
;; Definimos una intersección de prueba estable en el REPL
> (defparameter *interseccion* '(45 5 40))
*INTERSECCION*

;; Verificamos la duración total del ciclo
> (duracion-ciclo *interseccion*)
90
```

```
:: Solicitamos el diagnóstico de optimización vial
```

```
> (recomendacion-ciclo *interseccion*)
```

```
CICLO-OPTIMO-FLUJO-VEHICULAR-EFICIENTE
```

```
:: Proyección analítica horaria sin recursión
```

```
> (calcular-porcentajes-eficiencia *interseccion*)
```

```
(ROJO 50.0 AMARILLO 5.5555556 VERDE 44.444447)
```

```
:: Evaluación de estados mediante marcas temporales absolutas (Unix Epoch)
```

```
> (timer 0 *interseccion*)
```

```
ROJO
```

```
> (timer 44 *interseccion*)
```

```
ROJO
```

```
> (timer 47 *interseccion*)
```

```
AMARILLO
```

```
> (timer 85 *interseccion*)
```

```
VERDE
```

☒ Errores y correcciones en el RELP durante este bloque:

-

Bloque 4: Persistencia de Datos e Intermittencia

❖ BLOQUE 4:

☒ Persistencia de Datos:

La persistencia de datos refiere a la capacidad de un sistema de conservar información más allá del ciclo de vida de su ejecución. A diferencia de la memoria RAM, que es volátil y se libera al finalizar el proceso, los datos persistidos se almacenan en un soporte físico permanente como el disco, garantizando su disponibilidad en ejecuciones futuras.

☒ Objetivo:

Implementar un sistema de auditoría histórica mediante la persistencia de estados en archivos de texto, garantizando la trazabilidad del funcionamiento del semáforo. Se introduce una capa de seguridad preventiva mediante estados "intermitentes" como la explicada en el bloque 3, antes de cada transición principal.

☒ Implementación de persistencia (Capa de E/S):

Para cumplir con la inmutabilidad, se define una función de escritura que gestiona el archivo de registro.

```
;;; Naturaleza: Impura (Realiza efectos secundarios de escritura en archivo)
;;; Entrada: Lista de datos, String con la ruta del archivo
;;; Salida: T (en caso de éxito)
(defun informe (datos ruta-archivo)
  (with-open-file (stream ruta-archivo
                        :direction
                        :output
                        :if-exists :append
                        :if-does-not-exist :create)
    (format stream "~%Timestamp: ~A | Estado: ~A" (first datos) (second
datos))))
```

I Persistencia de escritura no destructiva

Para auditoría forense necesitás el archivo completo. Si hay un accidente de tránsito y el perito pregunta qué color tenía el semáforo hace dos horas, esa información la agrega :append al final sin borrar nada. Como seguir escribiendo en el mismo cuaderno.

☒ Integración lógica y auditoría:

La función procesar-estado-seguro actúa como el punto de entrada, integrando el cálculo de estado (bloque 3) Persistencia (bloque 4).

```
;;; Naturaleza: Impura (Llama a funciones de persistencia)
;;; Entrada: Entero (Timestamp), Lista de config, String de archivo
;;; Salida: Símbolo del estado calculado
(defun procesar-estado-seguro (time-exac configuracion ruta-archivo)
  (let ((estado-actual (timer-suegro time-exac configuracion)))
    ;; Se realiza la escritura en disco
    (informe (list time-exac estado-actual) ruta-archivo)
    ;; Se retorna el estado para uso del sistema
    estado-actual)))
```

La escritura en disco se concentra exclusivamente en la función informe, manteniéndola aislada de la lógica pura del sistema. De esta forma, funciones como timer-seguro conservan su transparencia referencial, mientras que la capa de I/O gestiona la comunicación con el mundo exterior sin contaminar los cálculos funcionales.

☒ Consideraciones de diseño:

- **Separación:** La función timer-seguro permanece pura, facilita su validación mediante pruebas unitarias. La lógica de escritura reside únicamente en el nivel superior, evitando efectos secundarios en los cálculos viales.
- **Robustez:** Se utiliza :if-exists :append para asegurar que el historial no sea sobrescrito, cumpliendo con los requisitos de auditoría forense del tráfico.

- **Manejo de estados:** Se ha integrado la interpretación de 3 segundos como un estado operativo válido, mejorando la seguridad ante fallos de sincronización.

☒ Validacion de la Persistencia forense:

Se demuestra que el archivo general (informe-ejecucion-semafor.txt) tiene el formato solicitado:

- **Acción:** ejecutamos la función procesar-estado-seguro con tres timestamps diferentes en el RELP.
- **Informe:** Se muestra como se ve el archivo de texto resultante:

Timestamp: 1715876400 | Estado: ROJO

Timestamp: 1715876405 | Estado: AMARILLO-INTERMITENTE

Timestamp: 1715876408 | Estado: AMARILLO

Bloque 5: Comparacion con el lenguaje Sheme

❖ BLOQUE 5: Concurrencia, paralelismo y condiciones de Bernstein.

☒ Objetivo:

Analizar el comportamiento del sistema de semáforos bajo un modelo de ejecución concurrente, identificar las condiciones de Bernstein aplicadas al problema y justificar por qué el paradigma funcional con inmutabilidad absoluta reduce los riesgos inherentes a la ejecución paralela.

☒ Condiciones de Bernstein

Las condiciones de Bernstein determinan cuándo dos procesos pueden ejecutarse en paralelo sin producir inconsistencias. Dos procesos pueden correr simultáneamente únicamente si los datos que escribe uno no son leídos ni escritos por el otro.

En un sistema de semáforos urbano esto es crítico porque múltiples intersecciones pueden estar ejecutándose al mismo tiempo. Si dos procesos intentan modificar el mismo estado compartido simultáneamente pueden producirse errores de sincronización, estados corruptos o comportamientos impredecibles.

☒ Desarrollo Técnico:

Las condiciones de Bernstein establecen que dos procesos P1 y P2 pueden ejecutarse en paralelo sin conflictos únicamente si se cumplen estas tres condiciones sobre sus conjuntos de lectura (R) y escritura (W):

(defun timer-seguro (time-exac configuracion)...)

;;;(P1) $\cap$ (P2) = $\emptyset$ ninguno escribe lo mismo que el otro.

(defun duracion-ciclo (configuracion) ...) ;;;(P1) no escribe lo que (P2) lee

(defun transicion (color-actual cambiar-a) ...) ;;;(P2) no escribe lo que (P1) lee

Si alguna de estas condiciones se viola, existe un conflicto de concurrencia.

☒ Posibles Conflictos:

La única zona de conflicto potencial es la función, informe que escribe en un archivo compartido. Este es el único punto del sistema donde las condiciones de Bernstein podrían violarse si dos procesos intentaran escribir simultáneamente en el mismo archivo.

```
(defun informe (datos ruta-archivo)
  (with-open-file (stream ruta-archivo
    :direction :output
    :if-exists :append
    :if-does-not-exist :create)
    (format stream "~%Timestamp: ~A | Estado: ~A"
      (first datos)
      (second datos))))
```

Si dos procesos llaman a informe simultáneamente sobre el mismo archivo:

P1 — informe ('(100 ROJO) "log.txt")

P2 — informe ('(200 VERDE) "log.txt")

$W(P1) \cap W(P2) = \{\text{log.txt}\} \neq \emptyset$ CONFLICTO

Ambos escriben en el mismo archivo simultáneamente, violando la primera condición de Bernstein. Esto puede producir líneas entrelazadas en el archivo:

;;;Resultado corrupto posible

Timestamp: 10Timestamp: 200 | Estado: VERDE

0 | Estado: ROJO

☒ Inmutabilidad absoluta en el Paradigma Funcional:

El modelo de inmutabilidad absoluta adoptado en el proyecto elimina la mayoría de los conflictos que las condiciones de Bernstein buscan evitar. Como ninguna función modifica estructuras de datos existentes sino que genera nuevo, dos procesos que calculen estados simultáneamente nunca compiten por el mismo recurso en memoria.

Bloque 6: Código completo y explicado de Scheme

❖ BLOQUE 6: Código Scheme y comparativas .

☒ Objetivo:

– Selector de color: